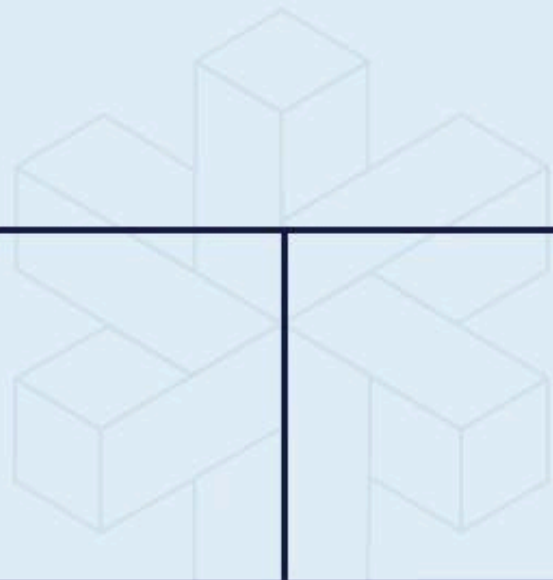




ClimatePros



Stream 1 Scope Document



Contents

1. Executive Summary	5
2. Objectives / Goals	7
3. Scope of the Document	8
4. Use Case A — Auto Dispatch	9
Purpose	9
Current State	9
Future State	9
A1 — Intake & Trigger	10
Purpose	10
Current State	10
Future State	10
Business Rules	11
Exceptions	12
Assumptions	12
Dependencies	13
Acceptance Criteria	13
A2 — Candidate Selection & Ranking	13
Purpose	13
Current State	13
Future State	13
Business Rules	14
Exceptions	15
Assumptions	15
Dependencies	16
Acceptance Criteria	16
A3 — Proposal & Assignment	16
Purpose	16
Current State	16
Future State	16
Business Rules	17
Exceptions	18
Assumptions	19

Dependencies	19
Acceptance Criteria	19
A4 — Escalation, Configuration & System Activity	19
Purpose	19
Current State	20
Future State	20
Business Rules	21
Exceptions	21
Assumptions	21
Dependencies	21
Acceptance Criteria	22
5. Use Case B — Closeout Copilot	23
Purpose	23
Current State	23
Future State	23
B1 — First-Screen Capture	24
Purpose	24
Current State	24
Future State	24
Business Rules	25
Exceptions	25
Assumptions	25
Acceptance Criteria	26
B2 — Field Suggestions & Prefill	26
Purpose	26
Current State	26
Future State	26
Business Rules	27
Dependencies	27
Acceptance Criteria	27
B3 — Note Standardisation	27
Purpose	27
Current State	27
Future State	27

Business Rules	28
Exceptions	29
Assumptions	29
Acceptance Criteria	29
B4 — Completeness Gate	29
Purpose	29
Current State	29
Future State	30
Business Rules	30
Exceptions	31
Assumptions	31
Acceptance Criteria	31
6. Use Case C — Parts Finder	32
Purpose	32
Current State	32
Future State	32
C1 — Part Prediction & Catalog Matching	33
Purpose	33
Current State	33
Future State	33
Business Rules	34
Exceptions	34
Assumptions	34
Dependencies	34
Acceptance Criteria	35
C2 — Source Finding & Ranking	35
Purpose	35
Current State	35
Future State	35
Business Rules	36
Exceptions	37
Assumptions	37
Dependencies	38
Acceptance Criteria	38

C3 — Needs-Parts Hand-off & Transfer Flow	38
Purpose	38
Current State	38
Future State	39
Business Rules	39
Exceptions	39
Acceptance Criteria	40
7. AI & Data Governance	41
8. Risks & Mitigation	43
9. Success Measures	45
10. Future Enhancements	46

1. Executive Summary

ClimatePros runs a 24/7/365 commercial refrigeration and HVAC field-service operation — roughly 900–1,000 technicians across 46 branches in 50 states, serving national grocery and retail customers such as Jewel, Target, Albertsons, and Safeway, among many others. The operation runs on **FieldJetX**, ClimatePros' field-service management platform: dispatchers manage incoming work on the web dispatch board, and technicians run their day — jobs, status updates, forms, and inventory — on the FieldJetX mobile app. Stream 1 builds on this platform and optimises how work moves through it; **it does not replace it.**

Three moments in the working day cost the most time today:

- **Dispatch.** On a typical weekday around 700 dispatches — each one a job to send a technician to a customer site — land in the **Ready for Dispatch** queue, and each is assigned by hand: the dispatcher researches the dispatch, weighs technicians by location, workload, and site familiarity, then phones candidates one by one to check availability before anything can be assigned — the same serial, phone-by-phone process at every desk.
- **Closeout.** When a job is done, writing up the closeout by hand takes a technician 10–15 minutes with no help from the app — and a write-up done at speed can still miss a photo, a part, or a detail the next technician at that site will need, so the office has to send the job back.
- **Parts.** When a repair needs a part that is not in any internal inventory, the search falls off the app entirely — the technician phones vendors or searches the web, often under emergency pressure. The app today gives the technician no view of where a part is available or how far away it is, so there is no way to head straight to the nearest source — and if nothing turns up nearby, the job ends in a return visit, breaking the **"one-and-done" first-visit-resolution** standard ClimatePros runs on.

Stream 1 attacks all three — giving dispatchers and technicians their time back, with **people in control of every decision** — through three AI capabilities built into the FieldJetX experience:

- **Use Case A — Auto Dispatch** — recommends a technician for an incoming dispatch and takes over the manual research and phone cascade, so the dispatcher confirms the assignment instead of chasing it.

- **Use Case B — Closeout Copilot** — helps the technician produce a complete, consistent write-up in less time than today's manual closeout.
- **Use Case C — Parts Finder** — helps the technician locate the parts a job needs and the nearest available source.

Four posture statements hold throughout:

1. **The system proposes; people decide.** Every automated action is reviewable, and a person makes every consequential decision.
2. **The decision core is deterministic and explainable.** AI is used only on the language edge — reading and interpreting free text such as notes and part descriptions — and stays out of the load-bearing decisions, which are handled by rules, not by a model.
3. **FieldJetX remains the system of record** — the single authoritative home for the data. The AI layer integrates through FieldJetX's APIs — extended only where a capability needs it — and nothing already working in the platform is replaced.
4. **Privacy-first.** ClimatePros' self-hosted private model is the priority runtime; commercial models are used only when a task requires it and only on data cleared for it, always within ClimatePros' data-privacy commitments to its customers.

This document defines **what Stream 1 will build, what is explicitly out of scope, and the assumptions that need ClimatePros' confirmation** — specified use case by use case, each broken into modules: the functional pieces — steps and features — that together deliver it.

2. Objectives / Goals

Stream 1 sets out to give ClimatePros' dispatchers and technicians their time back across the three tasks that consume it today — dispatching, job closeout, and parts sourcing — while keeping people in control of every decision and FieldJetX as the system of record. The goals below are the targets Stream 1 aims for; each target and how it is measured are confirmed with ClimatePros (see Success Measures).

- **Faster technician assignment.** Replace the manual research and one-by-one phone cascade with a ranked, one-tap proposal, so a dispatcher confirms an assignment in a single review step instead of working the phones for every job. (*Use Case A*)
- **Faster, more complete closeout.** Cut the time a technician spends writing up a job — today 10–15 minutes — by up to roughly half, while catching a missing photo, part, or note before submission rather than after the office sends the job back. (*Use Case B*)
- **More first-visit fixes on parts.** Bring the off-app parts hunt back into the app and show the nearest available source, so more jobs finish on the first visit and fewer end in a return trip. (*Use Case C*)
- **A costed path to the next phase.** Deliver the in-scope slice defined in each use-case section now, and gather the remaining requirements as a parallel discovery track — not a build commitment of this phase — so the next phase can be scoped and costed on evidence.

Every goal holds within the four posture statements the Executive Summary sets out — the system proposes and people decide, the decision core is deterministic, FieldJetX stays the system of record, and the model strategy is privacy-first. Each use-case section states its own measurable acceptance criteria and the baseline it is measured against.

3. Scope of the Document

This document defines the **functional scope** of Stream 1 — the business logic, rules, exceptions, assumptions, dependencies, and exclusions for what will be built.

It covers three use cases:

- **Use Case A — Auto Dispatch** — proposes a technician for a dispatch.
- **Use Case B — Closeout Copilot** — records what happened on the job.
- **Use Case C — Parts Finder** — sources the parts the job needs.

Each use case is broken into **modules** — the functional pieces (steps and features) that together deliver it — and every module is specified under the same structure: **Purpose** • **Current State** • **Future State** • **In Scope** • **Out of Scope** • **Business Rules** • **Exceptions** • **Assumptions** • **Dependencies** • **Acceptance Criteria** (any part with nothing to say is left out — an omitted part means it does not apply here, not that it is unexamined). Open items that need ClimatePros' confirmation are flagged within each module, under its Assumptions and Dependencies.

What this document does not define: detailed screen or UI designs, the technical architecture (covered by the separate architecture deliverable), or delivery timelines and effort estimates.

What stays as it is. Stream 1 is an AI layer on top of FieldJetX; it does not re-scope the rest of the platform. Everything else carries forward unchanged — how customer requests arrive and become dispatches, the dispatch board, every status other than the single Ready-for-Dispatch → Dispatched transition Auto Dispatch automates, the overnight scheduler, preventive-maintenance route planning, custom-forms administration, and inventory management. FieldJetX remains the system of record.

4. Use Case A — Auto Dispatch

Purpose

Automate the assignment of a new dispatch to the right technician — taking the manual research and one-by-one phone cascade off the dispatcher's plate — while keeping the dispatcher in control and able to step in at any point. It augments the dispatcher; it does not replace them: the system proposes and the dispatcher decides, the ranking is rule-based rather than AI, and FieldJetX stays the system of record.

Current State

A dispatch lands in the **Ready for Dispatch** queue and the dispatcher assigns it by hand: research the job, weigh technicians by location, workload, and site familiarity, then phone candidates one at a time to check availability until someone accepts. Availability can only be confirmed person by person, so assignment is a serial process whose speed depends on who picks up — and nothing records why a given technician was chosen.

Future State

With Auto Dispatch switched on for their area, the dispatcher lets the system do the legwork: each new dispatch that carries an in-scope priority is picked up, the eligible technicians are ranked, the job is offered to their phones in ranked order, and the first to accept is assigned — while the dispatcher watches **System activity** (a plain-language feed of everything the system does) and can override or take over at any moment. The ranking and its reasons are produced by fixed rules, not by an AI model.

Auto Dispatch is delivered as four modules, each specified in full below:

- **A1 — Intake & Trigger** — which dispatches the system acts on.
- **A2 — Candidate Selection & Ranking** — who is eligible, and in what order they are offered the job.
- **A3 — Proposal & Assignment** — offering the job to technicians and making the assignment.
- **A4 — Escalation, Configuration & System Activity** — the dispatcher's controls, fallback, and audit trail.

Who is involved.

- **Dispatcher** — turns Auto Dispatch on and off for their area, watches System activity, works the escalated queue, and can override or manually assign any dispatch at any time; a manual assignment always takes precedence.
- **Technician** — receives the job offer on their phone (and an availability check when already on a job), accepts or declines, and sequences their own day; the system assigns work but never reorders a technician's schedule.

The one automated change. Across all four modules, the system makes exactly one automated status change — **Ready for Dispatch** → **Dispatched**, and only when a technician accepts. Every other status change stays a human action.

A1 — Intake & Trigger

Purpose

Decide which dispatches the auto-dispatch engine acts on. Its entry point is the *classified* dispatch in the Ready for Dispatch column: it acts on dispatches that already carry one of the three in-scope priorities and offers them for assignment. It does not judge whether a dispatch "should" be dispatched — that judgement, and the classification itself, stay with ClimatePros.

Current State

Work arrives in the Ready for Dispatch queue already carrying a priority (shown on the board as a colour). The dispatcher reads each one and decides what to do with it — assign it, or route it elsewhere (a preventive-maintenance route, a construction/project job, an approval, a hold) — from experience.

Future State

When Auto Dispatch is on for the area, every in-scope-priority dispatch in Ready for Dispatch — both those already queued when it is switched on and new ones arriving while it stays on — is picked up and passed to ranking (A2). Dispatches carrying any other label are left exactly as they are today, for the dispatcher. The system reads the priority as given; it does not re-classify it.

In Scope	Out of Scope (this phase)
• Acting on in-scope dispatches in Ready for Dispatch while the area is on — both those already in the queue	• Any dispatch not carrying one of the three in-scope priorities — preventive-maintenance routes (PM),

In Scope	Out of Scope (this phase)
when it is switched on and new arrivals. - Recognising the Ready for Dispatch and Dispatched statuses from an agreed list per company and region, not a single hard-coded value. - Acting only on dispatches carrying one of the three in-scope priority levels — emergency, high, and low — as expressed by the board's priority tags and colours (the board carries tags such as Overtime Approved, High, and Normal); which tag maps to which level is confirmed with ClimatePros (Rule 2). A dispatch on any of these priorities is passed to ranking. - Checking whether an open work order already exists at the same customer location, so a technician already there can be preferred (feeds A2).	scheduled / time-committed work (LOCKED – SCHEDULED), special-equipment work (Special Eq.), and manager-approval or hold states (for example AM: Acct Mgr Approval, AM: DO NOT DISPATCH) — are not touched by the system; they remain the dispatcher's to handle as today. Deciding whether a dispatch is genuinely dispatchable, or re-classifying it. The system consumes the priority as given; it does not screen for "should this really be auto-dispatched," infer a different priority from the wording, or correct a mis-classified dispatch. Keeping the right work — and the right priority — in the queue stays part of ClimatePros' existing dispatch process.

Business Rules

- The entry point is the classified dispatch.** Auto Dispatch acts on dispatches carrying one of the three in-scope priorities. When the area is switched on, it considers the in-scope dispatches already sitting in Ready for Dispatch as well as every new one that arrives while it stays on.
- Three in-scope priorities; everything else is left alone.** The system acts on emergency, high, and low priority dispatches. Any dispatch carrying a different label — a preventive-maintenance route, a construction/project job, or another internal status — is out of scope and is neither picked up nor changed. *(The exact mapping of the board's colours/labels to these three priorities is confirmed with ClimatePros — see the sign-off worklist.)*

3. **Classification is ClimatePros', not the system's.** The system reads the priority a dispatch already carries and acts on it as-is. It does not classify, re-classify, or infer priority from free-text — so a dispatch tagged low with a "2-hour emergency" in its notes is handled as low, though it stays fully visible to the dispatcher, appears in System activity, and can be corrected or handled manually at any time. Keeping the priorities accurate stays part of ClimatePros' existing dispatch process.
4. **Status recognition.** "Ready for Dispatch" and "Dispatched" are not single values — they vary by company and region, and some dispatches still carry older variants. The system works from an explicit agreed list of which statuses count.
5. **Same-store preference.** If an open work order already exists at the dispatch's customer location and a technician is assigned or on site there, that technician is preferred for the new dispatch (carried into A2).

Exceptions

- A dispatch cancelled, held, re-prioritised, or merged before it is offered is not admitted; it re-enters under its new state (and, if its new priority is out of scope, is left for the dispatcher).
- A dispatch in a status the system doesn't recognise is left for the dispatcher — never acted on by a guess.

Assumptions

- ClimatePros keeps the queue disciplined: only work meant for auto-dispatch carries the three in-scope priorities, and the priority on a dispatch is the one the system should act on. *(Priorities are known to be inconsistent today — sometimes a "2-hour emergency" sits in the notes of a low-priority dispatch, and SLAs are not always recorded in a single field. This is recognised; the system acts on the priority as given and does not attempt to reconcile it.)*
- While Auto Dispatch is off, queued Ready-for-Dispatch work is handled as it is today (by the dispatcher and the existing overnight scheduler — ClimatePros' separate morning-scheduling initiative, out of scope here (see A4)); when it is switched on for an area, the system takes over the in-scope queue for that area. The two never both own the same dispatch — the engine owns the in-scope queue while the area is on, the scheduler runs while it is off, and the safe assignment step (A3), which reads a dispatch's committed status before acting, prevents any double assignment in an overlap.

Dependencies

- The mapping of the board's colours/labels to the three in-scope priorities (emergency / high / low).
- The list of statuses that count as Ready for Dispatch / Dispatched per company and region.

Acceptance Criteria

- A dispatch carrying one of the three in-scope priorities in Ready for Dispatch (area on) is handed to ranking – whether it was already queued at switch-on or arrived afterwards; a dispatch carrying any other label is neither picked up nor changed.
- The system acts on the priority as given and never re-classifies or infers a different one.
- A dispatch in an unrecognised status is left for the dispatcher.

A2 – Candidate Selection & Ranking

Purpose

Turn an eligible dispatch into an ordered list of technicians to offer it to – build the candidate pool, remove those who can't take it, and rank the rest so the best-placed technician is offered first. The ranking is produced by fixed, inspectable rules; no AI model chooses or orders technicians.

Current State

The dispatcher weighs candidates by location, current workload, and how well they know the site, drawing on personal knowledge – so the choice varies by dispatcher, and the reasoning is never recorded.

Future State

The system assembles the eligible technicians, scores each against agreed factors (the exact scoring logic is detailed in the technical document), and produces a fixed ranked order with a plain-language reason for each – ready to offer in A3. The same inputs always produce the same order and the same reasons.

In Scope	Out of Scope (this phase)
• Building the candidate pool: technicians on the service side of the	• Ranking on skill or seniority, or holding a senior technician in reserve – the

In Scope	Out of Scope (this phase)
business, within the configured area, available, and not over their workload limit; the same-store preferred technician (A1) placed first (for an emergency — ranked on speed — an on-site or same-store technician is already the fastest, so this preference is consistent with speed-only ranking). • Ranking that balances how soon a technician can arrive with how familiar they are with the site, with familiarity weighted more as urgency drops. • Priority handling driven by the classification the dispatch already carries — an emergency ranks on speed alone. • A fixed tie-break for exact ties. • A plain-language reason for each ranked technician.	structured competency data isn't reliably available today; site familiarity stands in for now. (<i>Future enhancement.</i>) • Automatically routing a call-back to the technician who worked it last. (<i>Future enhancement.</i>) • Widening the search area automatically when the pool is thin.

Business Rules

1. **The shortlist.** A technician is eligible when they are within the configured area (radius) and available. "Available" means a service technician (not construction/modernization staff), not on approved time off, and not already over the workload limit (Rule 6). There is no per-technician online/offline switch — availability is governed by the area toggle and the offer itself.
2. **Urgency comes from the dispatch's existing classification.** The system reads the priority already assigned to the dispatch and ranks accordingly — an emergency on speed alone, other work on a balance of speed and familiarity. It does not classify dispatches or infer urgency from free-text wording. (*Where a dispatch is mis-tagged upstream, it is handled at whatever priority it carries; correcting classification is out of scope this phase.*)

3. **Ranking factors.** (a) *Speed* — how soon the technician can be on site: time until free (for a technician already on a job, the ETA they give in A3; zero if they are free) plus travel time. (b) *Familiarity* — prior service visits to that customer location (construction/modernization visits excluded), recent rather than lifetime, weighted up as urgency drops.
4. **Ranking is a configurable weighting of speed and familiarity.** Each non-emergency priority carries a weighting the dispatcher sets on a configuration screen, so ClimatePros controls how the system behaves without a code change: an **emergency** looks for the closest technician — speed only, familiarity ignored — from the already-eligible pool, so it is the closest *qualified*, service-side technician, not simply the nearest; the emergency weighting is fixed and is not editable. Other priorities balance speed and familiarity, and that balance is adjustable (for example 80% speed / 20% familiarity). Whatever weighting is set, the ranking is repeatable and explainable, keeps genuinely close candidates close, and never discards a viable technician for being last in a tight field. The precise scoring math behind the weighting is detailed in the technical document;
5. **Tie-break.** When two technicians score equal, the order breaks by fewer minutes left on the current job, then fewest scheduled jobs, then a stable identifier.
6. **Workload limit.** The system won't offer a non-low-priority job to a technician already holding one active and one scheduled job; they re-enter the pool when the active job completes. Emergencies override the limit; low-priority jobs may be assigned beyond it; the same-store preferred technician is exempt. An emergency, though, is never stacked on a technician who already holds an active or pending emergency dispatch: while a technician still has an emergency to start, they are not offered another emergency, and other (non-emergency) offers to them are held back — so a live emergency is never undercut by easier work.

Exceptions

- No eligible technician → the dispatch goes straight to the escalated queue (A4) with the reason.
- A site with no service history → familiarity is simply low for everyone; ranking leans on speed rather than failing.

Assumptions

- A technician's upcoming jobs are an ordered list of work, not fixed appointment times.
(If they carry promised time slots, the logic is revisited.)
- Technician and truck locations are available to the system.

Dependencies

- Live technician/truck location (shared with Use Case C; an open location-accuracy issue affects some technicians — until resolved they are ranked from their last known position, marked "not live").
- A traffic-aware travel-time source.
- The fields that distinguish service from construction/modernization technicians and visits.
- The starting weightings per priority and the familiarity lookback window.

Acceptance Criteria

- The same inputs always produce the same order and the same reasons; no AI takes part in selection, ranking, or reasons.
- A technician off the service side, outside the area, on time off, or over the limit doesn't appear; the same-store technician appears first.
- An emergency ranks on speed alone; other work reflects both factors, per the configured weighting.
- Exact ties resolve by the fixed tie-break.

A3 — Proposal & Assignment

Purpose

Take the ranked list and turn it into an assignment — check availability where needed, offer the job down the order, and on the first acceptance make the one automated status change. A job no one takes falls to the dispatcher (A4). The guarantee that matters: a dispatch is never assigned to two people.

Current State

The dispatcher phones candidates one at a time to check availability and offer the job — a serial process whose speed depends on who answers.

Future State

The round of calls becomes a one-tap offer on the technician's phone that moves down the ranked list on decline or timeout. A busy technician is first asked how long until they're free (their ETA); the job is then offered in ranked order with a short stagger so the top technician gets a genuine head start. The first to accept is assigned, the dispatch moves to Dispatched, and every other open offer is withdrawn.

In Scope	Out of Scope (this phase)
• An availability check — a quick "how long until you're free?" (the technician's ETA) — for a technician already on a job. • The ranked offer: one-tap accept notifications sent down the order with a short stagger, more than one able to be live at once, first-accept-wins. • Offering directly to a technician already on site at the location, with no stagger. • A safe assignment step that guarantees a dispatch is only ever assigned once — across both a technician's acceptance and a dispatcher's manual assignment. • The single automated status change: Ready for Dispatch → Dispatched, on acceptance only. • Re-checking a technician is still eligible immediately before each offer. • Notifying the dispatcher when a technician accepts.	• Broadcasting one job to everyone at once (first tap wins) — rejected: it destroys the ranking and rewards whoever is glued to their phone. • Predicting how long a repair will take — availability times are always technician-entered. • Sourcing a second technician or helper — the system assigns the primary technician and flags that a helper may be needed. (<i>Future enhancement.</i>)

Business Rules

1. **Availability check.** A busy technician is asked how long until they're free (their ETA) — once, or at most twice at a set interval (the number of prompts and the interval are set in configuration) — after they've had time to assess their current job. The answer supplies the "time until free" used in ranking (A2) and is discarded when it goes stale.
2. **The offer moves down the ranked order** with a short stagger between technicians; offers already sent stay live, so more than one technician may hold a live offer for the same job. This is not a broadcast — ranking is preserved by the timing.
3. **First acceptance wins.** The moment a technician accepts, the dispatch is assigned and every other open offer is withdrawn. Each dispatch has a maximum total offer time and escalates to the dispatcher if it's reached; emergencies use a compressed, repeating alert.
4. **On-site short-circuit.** A technician already on site at the location is offered the job directly, with no stagger.
5. **Safe assignment.** Assignment is atomic — a dispatch can only ever be assigned once, and a technician's acceptance and a dispatcher's manual assignment can never both take effect. *(This needs a safe write path in FieldJetX that the app doesn't provide today — see the sign-off worklist; it is one of two hard prerequisites.)*
6. **One automated change only.** Ready for Dispatch → Dispatched, on acceptance (or a dispatcher's manual assignment through the same safe step). No other status changes automatically.
7. **Fresh check before each offer.** The ranked order is fixed when the offer round starts, but each technician's eligibility is re-checked just before their offer — anyone who has since become ineligible under A2's rules (now unavailable, or over the workload limit — with the same emergency-override and same-store carve-outs) is skipped.

Exceptions

- Two technicians accept at the same instant → only one is assigned; the other is told it's taken.
- A dispatcher manually assigns while the job is being offered → the manual assignment wins and open offers are withdrawn.
- No response within the window → treated as unavailable; the offer moves on.
- The dispatch is cancelled, held, re-prioritised, or merged mid-offer → open offers are withdrawn and it's re-evaluated; a cancellation beats a simultaneous acceptance.

- Mobile notifications or FieldJetX briefly unavailable → no new offers; the dispatch is surfaced, and offer state is held so a restart resumes cleanly rather than double-sending.
- A technician loses connectivity → they don't receive the offer; after the window it moves on.

Assumptions

- The on-site check-in event is available to detect arrival; where it isn't, truck location is the fallback.
- Technician training and encouragement to answer promptly is ClimatePros-provided; the system provides the reminders.

Dependencies

- **A mobile push channel (sign-off critical).** A server-to-phone push channel to be stood up for the offers and reminders (today the app uses local notifications).
- Who builds the mobile changes (the offer screen, availability prompt, push).
- The stagger, offer windows, total offer ceiling, and on-site geofence.

Acceptance Criteria

- A dispatch is never assigned to two technicians; a manual assignment and an acceptance never both take effect.
- The offer reproduces the stagger + first-accept-wins behaviour; the first acceptance withdraws all other offers.
- The one status change fires only on acceptance (or a manual assignment through the same safe step).
- A technician who has become ineligible before their offer is sent is skipped.

A4 — Escalation, Configuration & System Activity

Purpose

Give the dispatcher the controls, the fallback, and the record — where an unassigned dispatch lands for a person, where the area's behaviour is set, and where every automated action is narrated. Escalation is a designed outcome, not a failure; the only unacceptable outcome is a wrong assignment.

Current State

The dispatcher is the fallback, the configuration (their judgement), and the only record of why an assignment happened – which lives in their head.

Future State

Anything the offer round can't resolve lands in the escalated queue with the ranked list, each technician's outcome, and the reasoning; the dispatcher assigns it, adjusts and re-runs, or re-queues. The dispatcher sets the area's weights, windows, area radius, and switch without a redeployment, and watches System activity throughout. At day's end the switch goes off (or an agreed dispatch-hours window turns it off), and later arrivals are handled the way after-hours work is handled today.

In Scope	Out of Scope (this phase)
• The escalated queue: unresolved dispatches with the ranked list, per-technician outcome, and reasoning; assign / adjust-and-re-run / re-queue. • Full dispatcher override at any point. • Per-area configuration (the speed-vs-familiarity weighting for the non-emergency priorities – the emergency weighting is fixed at speed-only, offer/availability windows, offer ceiling, area radius, reminder intervals, the switch) – set per area down to a branch, effective on the next dispatches without a redeployment. • A single on/off switch per board plus a configured daily dispatch-hours schedule per board that switches the system off outside those hours. • System activity: a live, plain-language, templated feed of every action,	• After-hours / weekend / on-call dispatching – the switch assumes a dispatcher on the board; after-hours work is handled as today. (<i>Future enhancement.</i>) • Reordering a technician's day – the system assigns; the technician sequences. • Integrating with the overnight scheduler – the two stay independent, sharing the FieldJetX database as the single source of truth. • Offering across a neighbouring area, per-site technician preferences, and site access-hour gating – each needs data the system can't yet read. (<i>Candidate future enhancements.</i>)

In Scope	Out of Scope (this phase)
extending FieldJetX's existing per-dispatch history.	

Business Rules

1. **Escalation is a designed outcome.** A dispatch reaches the escalated queue only after its candidates are exhausted, time out, or are excluded (or it's routed there directly); the view shows every technician's outcome, and a manual assignment there uses the same safe step as an acceptance.
2. **Manual assignment always wins** over an in-flight offer round and withdraws its open offers.
3. **Configuration is data, not code** — changes take effect on the next dispatches with no redeployment; a branch setting overrides the area default.
4. **One switch per board, plus an auto-off.** The switch is shared per board (last action wins, shown with who set it and when); an agreed dispatch-hours window turns it off daily so after-hours safety never depends on someone remembering. (Throughout Auto Dispatch, an "area" is a dispatcher's board; the per-company/region status list in A1 is a separate, data-level scoping, not another control boundary.)
5. **System activity is templated narration** of every action — no AI writes it.

Exceptions

- Switched off while a dispatch is still being offered → new pickups stop; in-flight offers finish or expire, and anything landing in a now-closed area is flagged as an unattended hand-off.
- A dispatch that shouldn't have been auto-assigned reaches a technician → the dispatcher can withdraw or reassign at any time.

Assumptions

- A baseline of today's dispatch volume and assignment time is captured on the pilot area before rollout.

Dependencies

- Whether the system can write into FieldJetX's existing per-dispatch history.
- Who else may view System activity, and any retention requirement.

- Confirming "area" = one dispatcher's board.

Acceptance Criteria

- A dispatch reaches the escalated queue only after its candidates are exhausted (or it's routed there); the view shows every outcome.
- A configuration change takes effect on the next dispatches with no redeployment; with the switch off, nothing is auto-processed.
- Outside the dispatch-hours window, no new offer rounds start even if the switch reads on.
- Every automated action produces a templated System activity entry with no AI authorship.

5. Use Case B — Closeout Copilot

Purpose

Help the technician close out a job faster and more consistently on the FieldJetX mobile app — capturing the essentials up front, offering one-tap suggestions and prefill in the fields, standardising the write-ups so the same fix reads consistently, and flagging anything incomplete before it is submitted. **The AI proposes; the technician reviews, edits, and submits.**

Current State

During and after a job the technician fills in forms and writes the closeout by hand — 10–15 minutes, and inconsistent: send ten technicians to the same "case running warm / replaced a fan motor" and you get ten different write-ups. The office frequently sends jobs back for a missing photo, thin notes, a missing part, or an incomplete form.

Future State

The technician sets the relevant closeout status and lands on a single first screen: pick the issue type and Summary Code (the Priority is shown, auto-filled from the dispatch), and describe — in their own words, at whatever length they choose, typed or dictated (voice, beta) — the problem and what they did. From that, the Copilot standardises the write-up into a consistent, house-style note — using only what was entered, not composing new content. The technician then fills any forms the job needs — with one-tap suggestions and prefill in the fields — enters time and parts, and a soft completeness check flags anything the office would otherwise bounce back, before submission. The office note is optional; when the technician adds internal detail, the Copilot standardises that the same way — it authors neither note.

Closeout Copilot is delivered as four modules:

- **B1 — First-Screen Capture** — the input the closeout write-up is built from.
- **B2 — Field Suggestions & Prefill** — one-tap suggestions and prefill in the relevant fields.
- **B3 — Note Standardisation** — the entered account, standardised into consistent notes.
- **B4 — Completeness Gate** — catch the office's bounce-back reasons before submission.

Who is involved.

- **Technician** — gives the account, reviews and edits the standardised notes, adds an office note if needed, resolves completeness flags, and submits.

The AI boundary. AI is used only on the language edge — standardising what the technician entered into consistent notes (rewording for consistency, never inventing), and reading entered text for the completeness check. It standardises the notes — the customer note, and the office note when one is added — but authors neither. Field suggestions and prefill, the completeness checks, and the data scrub are deterministic.

BI — First-Screen Capture

Purpose

Capture, on the first screen the technician sees after setting the relevant closeout status, the few structured fields and the free-text account the write-up is built from — while it is fresh, in one place.

Current State

The technician writes the closeout at the very end, by hand, in whatever words they choose; the same fix comes out differently every time.

Future State

Setting the relevant closeout status opens the **Complete Call** screen: the issue type and **Summary Code** (ClimatePros' standard closeout code; with the **Priority** shown, auto-filled from the dispatch), and a single box — *"What was the problem and what did you do?"* — which the technician types or dictates (voice, beta), at whatever length they choose. The most common values appear as one-tap suggestions in the structured fields; anything else is a manual pick. This screen is the source for note standardisation (B3).

In Scope	Out of Scope (this phase)
• The first-screen capture, shown when the technician sets a relevant closeout status (not during the job). • The free-text "what was the problem and what did you do" box — as much or as little as the technician wants to	• Prompting for the note while the technician is still working the job — a future phase; this phase captures when the relevant status is set.

In Scope	Out of Scope (this phase)
write; the source the notes are standardised from. • Voice dictation into that box — declared beta; typing remains primary. • The most common one-tap suggestions shown in the structured fields on this first screen (issue type / Summary Code), so the technician taps rather than types; suggestions inside the forms come later (B2).	• Heavy AI on the structured fields — the most common options are offered; more effort is not worth the value. • Voice general availability (beyond the declared beta).

Business Rules

1. **Capture when the closeout status is set.** The note is captured on the first Complete-Call screen, after the technician sets a relevant closeout status — not mid-job.
2. **The technician's account is the source.** The free-text box — the problem and what was done, in the technician's own words, at whatever length they choose — is the narrative the notes are standardised from; the structured entries (forms, parts, time) provide supporting context.
3. **Voice is additive.** Voice dictation never replaces typing, never blocks closeout when unavailable, and produces text the technician sees and edits before use.
4. **Keep the suggestions simple.** The structured fields offer the most common one-tap suggestions; the technician picks anything else manually.

Exceptions

- No connectivity at the site → capture works offline and syncs with the rest of the closeout before submission.

Assumptions

- The Priority shown is the one already on the dispatch (auto-filled, for confirmation — not set here).

Acceptance Criteria

- The capture appears when a relevant closeout status is set, on the first screen, not during the active job.
- The free-text box accepts typed or dictated input of any length and is the source passed to note standardisation.
- Voice being unavailable never blocks closeout; typing is always available.

B2 — Field Suggestions & Prefill

Purpose

Cut the typing in the fields the technician fills — show the likely values as one-tap suggestions beneath the relevant fields, and prefill what's already known, so the technician taps instead of typing.

Current State

The technician opens each form and types values the system already knows (equipment, customer, dispatch context) by hand.

Future State

As the technician works a form, the most likely values appear as one-tap suggestion chips in the relevant fields — from equipment, customer, dispatch context, and history — and known values prefill directly; the technician confirms or edits. The technician chooses which forms to open; the one form the system insists on — the refrigerant form when refrigerant is mentioned — is enforced by the completeness gate (B4).

In Scope	Out of Scope (this phase)
• One-tap suggestion chips shown in the relevant fields, from equipment, customer, dispatch context, and history. • Prefilling a field with a known value. • Suppression when confidence is low — no chip rather than a wrong guess.	• Silent prefill that fills a field without a tap. • Deciding which forms a job needs — the technician chooses (the refrigerant-form requirement is enforced in B4). • Interpreting the content of attachments.

Business Rules

1. **Suggest, never silently fill.** Suggestions appear as one-tap chips in the relevant fields; nothing is committed without the tap.
2. **Prefill known values** into the field the technician is on.
3. **Weak signal degrades to silence.**

Dependencies

- A field-by-field whitelist — which fields are safe to suggest and prefill.
- Continued read access to forms and history.

Acceptance Criteria

- A relevant field shows one-tap suggestion chips from known data; nothing commits without a tap.
- A low-confidence field shows no chip.

B3 — Note Standardisation

Purpose

Take what the technician entered (the first-screen free-text account as the narrative, with forms, parts, and time as structured context) and standardise it into consistent notes — the same fix written up consistently, in a house style — using only what was entered, so nothing is invented and the technician doesn't hand-write the final wording.

Current State

Every technician writes the notes by hand; wording and quality vary widely, and customer-facing text sometimes carries internal detail it shouldn't.

Future State

The Copilot standardises the customer-facing note from what the technician entered, in a consistent house style; the technician reviews and edits it, and it is signed on site when the work is complete. The office note is optional — the technician adds it only when there is internal detail the customer shouldn't see — and when they do, the Copilot standardises that the same way. It authors neither note. The technician can also tap a small **"enhance" icon** on selected free-text fields to have the AI tidy and standardise what they wrote in that one field. The injury / active-leak path is left untouched.

In Scope	Out of Scope (this phase)
- Standardising the customer-facing note from the entered account and data, in a consistent house style. - Standardising the office note the same way, when the technician adds one. - A per-field "enhance" action on selected free-text fields — the technician taps it and the AI tidies/standardises what they wrote in that field; it appears only on the fields configured for it, not on all fields. - A deterministic scrub so the customer note carries no prices, rates, or internal part numbers. - The technician's review and edit of the standardised notes before use.	- Authoring note content the technician did not provide — the AI standardises what is entered; it does not write from scratch or embellish. - Producing a customer-facing note when there is no completed work to present. - The injury / active-leak path — untouched. - Interpreting the content of attachments.

Business Rules

- The AI standardises; it does not author.** It reworks what the technician entered — the customer note, and the office note when one is added — into consistent, house-style wording, never adding information the technician didn't provide.
- The customer-facing note is for completed work;** the office note can be standardised on any closeout where the technician adds one.
- Never invent.** The notes contain only what is traceable to the technician's account and the record; uncertain points are flagged, not asserted.
- Consistent house style — standardise, don't predict.** The same fix reads the same way regardless of who wrote it. For both the customer note and the office note, the Copilot only standardises and refines the wording — it is not predictive, and it never changes the context or meaning of what the technician entered.

5. **Per-field enhance is opt-in.** The "enhance" icon appears only on selected free-text fields; the technician taps it to standardise that field's text and reviews the result — it never rewrites a field on its own.
6. **Customer-facing hygiene.** The customer note carries no prices, labour/overhead rates, or internal part numbers. (*The scope of the pre-model-call data scrub, and how it applies across the private and commercial runtimes, is set in AI & Data Governance.*)
7. **Reviewed before use.** Standardised (and dictated) text is shown for the technician's edit before it is submitted.

Exceptions

- The AI runtime is unavailable → closeout is never blocked; the technician writes the notes manually and signs on site as today.

Assumptions

- The historical corpus of good write-ups is usable to shape the house style; examples are curated with ClimatePros.

Acceptance Criteria

- The notes are standardised from entered data only, with nothing invented; the AI authors neither note.
- The customer note carries no prices, rates, or internal part numbers.
- The per-field enhance acts only on the field the technician taps and only on configured fields.
- With the runtime unavailable, the technician can still write and sign the notes; closeout is not blocked.

B4 — Completeness Gate

Purpose

Catch, before submission, the gaps the office would otherwise bounce the job back for — kept deliberately simple, so the review screen never "feels like too much."

Current State

Completeness is judged after the fact by the office; a job with a missing photo, thin notes, a missing part, or an incomplete form is sent back, and the technician re-opens it.

Future State

At the end of closeout, a soft review screen shows a per-section status — **green** complete, **yellow** present-but-thin, **red** required-and-missing — checked against **ClimatePros' own closeout-defect categories**, not a new definition of "incomplete." It reads what the technician entered (not the content of images): if the note says a leak was found, it expects a leak photo and flags its absence. The technician resolves or acknowledges each flag and submits; nothing is hard-blocked (with the one exception of a missing refrigerant form).

In Scope	Out of Scope (this phase)
- A soft, simple review screen with per-section green / yellow / red status against ClimatePros' existing defect categories. - Note-driven expectations — e.g. a leak in the note → expect a leak photo; presence is checked from the note text, not by analysing the image. - The refrigerant-form check — refrigerant mentioned → the refrigerant form must be present (the one hard requirement). - An empty-parts check — where the account names a part or a repair action but no part is listed, flag to add it or confirm none.	- Hard-blocking submission — the gate prompts and never blocks, with the single exception of the refrigerant form. - Analysing the content of photos/attachments (presence is checked, not content). - Telling the technician which non-refrigerant forms a job requires (detecting a required form never opened is a Future Enhancement). - A heavy or cluttered review screen.

Business Rules

- The gate checks what the office already rejects for.** Built against ClimatePros' existing closeout-defect categories; each maps to a deterministic check, an AI-assisted flag, or "not detectable at closeout." A "present-but-thin" (yellow) flag means a note missing one of the agreed summary must-haves (B3), not a subjective judgement.

2. **Prompts softly (one hard exception).** Flags are resolved or explicitly acknowledged; the gate explains what is missing in plain language and never hard-blocks submission — except the refrigerant form (Rule 4), which is required to submit.
3. **Photo expectations come from the note.** A stated leak (or similar) with no photo raises a flag — derived from the note text, not from analysing the image.
4. **Refrigerant form — the one hard requirement.** Whenever refrigerant is mentioned, the refrigerant form must be present to submit; this is the single check the gate enforces (it is an EPA-tracked record), while every other flag is a soft prompt.
5. **An empty parts list is itself checked** where the account names a part or a repair action but no part is listed.
6. **Keep it simple.** The review screen shows only what needs attention.

Exceptions

- No connectivity → the completeness check runs when connectivity returns, before submission.
- A form definition changed mid-job → definitions are re-read at form open, mirroring current FieldJetX behaviour.

Assumptions

- Customer procedures: a few require the technician's sign-off; the rest are shown as read-only reference. *(The intent is to reduce the roughly ten procedures to the two or three that genuinely need validation.)*

Acceptance Criteria

- Each covered category maps to a deterministic check, an AI flag, or "not detectable"; any flag is resolvable by fix or acknowledgement; submission is never hard-blocked except for a missing refrigerant form.
- A stated leak with no photo raises a flag; refrigerant mentioned with no refrigerant form blocks submission until the form is added; a part named in the account with none listed raises a flag.
- The review screen shows only items needing attention.

6. Use Case C — Parts Finder

Purpose

When a job needs a part, help the technician find the parts it needs and the nearest place to get each — on top of the existing inventory surfaces. Two capabilities: **predict** the parts a job likely needs (from the dispatch context and similar past jobs) and **source** each confirmed part (own truck → nearby owned stock → external vendor), ranked by how close it is. **The system proposes; the technician confirms every part and chooses every source.** Collection uses the existing transfer flow; procurement is unchanged.

Current State

When a job needs a part that isn't on the truck, the technician phones around or searches the web, often under emergency pressure; the app shows no view of where a part is or how far away it is. If nothing is found nearby, the job ends in a return visit.

Future State

On the add-parts screen, the system suggests the parts the job likely needs; the technician confirms, adds, or removes each. For every confirmed part it shows where the part is actually available — own truck, a nearby technician's truck, a ClimatePros warehouse, or a United Refrigeration branch — grouped by ownership and ordered by how close it is, with a one-tap call to verify before driving. Whatever isn't covered pre-fills the existing needs-parts request for the office to order. The one AI step is reading the symptom; the prediction (an aggregation over historical parts-used data), the matching to catalog parts, and the ranking are deterministic.

Parts Finder is delivered as three modules:

- **C1 — Part Prediction & Catalog Matching** — suggest the parts a job likely needs, and match loose references to the catalog part.
- **C2 — Source Finding & Ranking** — for each part, where it is and the nearest place to get it.
- **C3 — Needs-Parts Hand-off & Transfer Flow** — what isn't covered flows to the office; collecting internal stock uses the existing (manual) transfer flow.

Who is involved.

- **Technician** — confirms, adds, or removes every predicted part; verifies each non-own-truck source by a one-tap call before driving; performs every status change; records any internal-stock transfer in the existing transfer flow.

The AI boundary. AI is used for one small step — reading the free-text call reason / root cause to extract the symptom, which is then matched against the historical parts-used data. Everything else — the aggregation over similar past jobs, the matching to the catalog part, the ranking by ownership and distance, and the shortfall allocation — is deterministic. Nothing is reserved and nothing commits until the technician acts.

C1 — Part Prediction & Catalog Matching

Purpose

Suggest the parts a job likely needs — from the dispatch context and the parts used on similar past jobs — and match a loose or legacy part reference to the one canonical catalog part, so the technician starts from a confirmable list rather than a blank search.

Current State

The technician works out the parts from memory and manual search; there is no suggestion from history, and loose or legacy part names don't reliably map to the catalog.

Future State

On the add-parts screen, the system suggests the parts this job likely needs — a deterministic aggregation of the parts actually used on similar past jobs, ranked by how often they were used, with the symptom matched to that history by the one AI step. The technician confirms, adds, or removes each. A loose or legacy reference is matched to the canonical catalog part with a confidence; below confidence, the system asks rather than guesses.

In Scope	Out of Scope (this phase)
• Part suggestions on the add-parts screen, from dispatch context and historical parts-used data (deterministic aggregation; AI only for symptom matching); every suggestion confirmable and removable; no	• Predicting non-catalog parts — parts-used history is catalog-only by design; non-catalog needs go through the needs-parts / office lane (C3). • Auto-committing or reserving any predicted part.

In Scope	Out of Scope (this phase)
auto-commit; weak signal suppressed rather than shown. Matching a fuzzy / legacy part reference to the canonical catalog part with a confidence; below confidence, the system asks.	Exposing per-equipment parts history in the app (a Phase-2 endpoint).

Business Rules

- Predict, confirm, then source.** No source query (C2) runs for a part the technician has not confirmed; manual add and delete are always available.
- History drives prediction; the model never invents part numbers.** Suggested parts must exist in the catalog; the AI's role is matching the symptom to history, not generating identifiers. A part below the historical-support threshold is suppressed.
- The equipment on the dispatch is a lead, not a fact.** Customers often select the wrong asset when raising the work order, so the named equipment is treated as unconfirmed; the technician can correct it and prediction re-runs against the corrected asset. Where the asset can't be confidently matched, the system suggests at reduced confidence rather than asserting.
- Match, don't guess.** A fuzzy or legacy reference is matched to the catalog part with a confidence; below the threshold, the system asks the technician rather than picking.

Exceptions

- No similar history, or nothing clears the threshold → an empty state; the manual add-parts path is untouched.
- A reference below match confidence → the system asks rather than guesses.
- Confident history against a likely-wrong asset → prediction does not assert; the technician corrects the equipment and prediction re-runs.

Assumptions

- Parts-used history is catalog-only by design; non-catalog needs go through the needs-parts lane.

Dependencies

- Read access to the catalog, cross-references, inventories, and historical parts-used data.
- An offline backtest of prediction quality before rollout.

Acceptance Criteria

- Every suggested part exists in the catalog; identical inputs return the same ranked suggestions with the same confidence basis; below-threshold parts are not shown.
- A confirmed part is required before any source query runs; manual add / delete is always available.
- A likely-wrong asset is not asserted against; correcting it re-runs prediction.

C2 — Source Finding & Ranking

Purpose

For each confirmed part, show where it is actually available and the nearest place to get it — own truck, nearby owned stock, or an external vendor — ranked owned-first and nearest-within-tier, so the technician can head straight to the closest source without leaving the app. **Cost is not shown and is not a ranking input.**

Current State

The technician phones around or searches the web; the app gives no view of where a part is available or how far away it is.

Future State

For a confirmed part, the system shows every source that carries it, grouped into three ownership tiers and ordered by travel distance within each tier; the technician verifies a non-own-truck source by a one-tap call before driving. Whatever isn't covered flows to the needs-parts hand-off (C3).

Tier	Sources	Why
0 — On the truck	the technician's own truck	zero drive; already paid for
1 — Other owned stock	nearby technicians' trucks · branch-visible ClimatePros warehouses	already paid for; a short drive or a hand-off

Tier	Sources	Why
2 — External vendor	United Refrigeration branches (live)	bought new; only for what owned stock can't cover

Table 1. Parts source tiers

In Scope	Out of Scope (this phase)
- Per-part availability across the three tiers, grouped by source, with quantity and distance. - Owned-first, nearest-within-tier ranking, with a single configurable cross-tier buffer (default: strict owned-first) that can let a dramatically closer vendor outrank a much-farther owned source. - Tap-to-call verification — one-tap dial for nearby trucks and United branches; a part is not treated as sourced until the technician confirms after the call. - Shortfall allocation — a greedy fill of the needed quantity across sources by tier then distance; any remainder flows to C3. - United as the first vendor behind an interface additional vendors can plug into without redesign.	Cost-based ranking / price optimisation — no price is shown and cost is not a ranking input; cost tie-breaking is a Phase-2, server-side enhancement. - Reserving, holding, or purchasing stock at any source — availability is informational plus tap-to-call; no writes to any external system. - Cross-branch warehouse collection — nearer out-of-branch stock is flagged for the office route, not ranked as collectable. - The shopper / runner delivery program; vendors beyond United; geocoding of non-integrated suppliers; warehouse / inventory data correction (mis-stock events are surfaced, not fixed).

Business Rules

- Owned-first, distance-within-tier.** The needed quantity is filled by exhausting each ownership tier by travel distance before the next; a vendor is proposed only for what owned stock can't cover.

2. **Travel time over distance where available;** plain distance is the accepted fallback, flagged in the ordering.
3. **The buffer is the one cross-tier knob.** A single configurable buffer governs whether a much-closer vendor may outrank a much-farther owned source; the Phase-1 default is strict owned-first.
4. **Verified by call; nothing is reserved.** Truck and vendor stock counts are leads, not promises; the flow requires the confirming call before treating a part as sourced. No hold is placed on any source.
5. **United data is live.** Quantity and distance come from the vendor gateway per request; a stale or failed response degrades that source to "unknown," never to an invented number.
6. **A nearby truck is a moving, shared source.** Travel to a truck uses its last-known position with the age shown; a truck being called elsewhere is shown "also being called"; only stock-carrying vehicles appear.
7. **The warehouse tier respects the technician's existing branch visibility.** Stock is scoped to the warehouses the technician is already entitled to see; a nearer out-of-branch warehouse is flagged for the office route, not silently dropped.
8. **Cost is not a technician-facing input.** No price or cost appears anywhere in the technician's flow; which vendors are eligible at all is a server-side configuration ClimatePros controls.

Exceptions

- United gateway down or slow → United sources show "unknown"; the internal tiers are unaffected.
- No traffic data for a route → that pair ranks by plain distance, flagged.
- Nearby technician unreachable, or stock wrong on arrival → the source list re-opens with that source demoted; a coordination failure is logged as a coordination event, a genuine discrepancy as a mis-stock signal.
- Nearest internal stock is out of branch → the technician is not sent to it; it is flagged for the office route.

Assumptions

- The United gateway contract remains as verified (branch geo, distance, quantity, branch phone) with production-suitable rate limits and credentials.

- Supplier location data may already exist in the platform, so distance ranking over internal supplier data is a data join, not a geocoding build (to be verified).
- Truck and warehouse inventory data is current enough to act on with call-verification; known accuracy issues are tolerated in Phase 1 with mis-stock events logged.
- The location inputs are the same feed Use Case A uses (a shared dependency and shared defect with Use Case A).

Dependencies

- United gateway production credentials and rate limits.
- The unified vendor-middleware roadmap, so the Phase-1 abstraction aligns with the planned multi-vendor layer.
- Buffer semantics and tier-ordering confirmation (the owned-first default).
- The GPS-locator defect and live-location coverage (shared with Use Case A).

Acceptance Criteria

- Each confirmed part shows every source that carries it, grouped by tier, with quantity and distance; a source with no data shows "unknown," never dropped silently.
- Identical inputs produce an identical ranked order and allocation; owned-first holds under the default buffer; no cost value appears anywhere in the technician's flow.
- A part is not treated as sourced until the technician confirms after the verifying call; nothing is reserved.

C3 — Needs-Parts Hand-off & Transfer Flow

Purpose

Hand whatever the internal tiers can't cover to the office to order, and let the technician record any internal stock they collect through the existing transfer flow so inventory stays accurate — without changing procurement, and without changing that transfer step.

Current State

When a part must be ordered, the technician phones the office. When a technician collects a part from another source — for example a nearby technician's truck — they record it manually in the app's transfer flow: they open the transfer flow, select the source (the truck the part came from), select the part they received, and submit, so the backend inventory adjusts. This is a manual step today.

Future State

Any confirmed quantity not covered by owned stock pre-fills the existing needs-parts request; the technician sets the status and the office orders. When the technician collects internal stock, they record it exactly as they do today – manually, in the existing transfer flow (open it, select the source, select the part, submit; inventory adjusts). Parts Finder routes the technician to the part; recording the transfer stays the technician's manual step and is unchanged.

In Scope	Out of Scope (this phase)
• Pre-filling the existing needs-parts request with the uncovered quantity; the status change stays the technician's action. • The existing manual transfer flow, unchanged – Parts Finder points the technician to the source; recording the collected transfer (open the transfer flow, select the source, select the part, submit) remains the technician's manual step exactly as today.	• Automating the transfer flow – recording a collected transfer stays manual, as today, in both this phase and the future scope. • The procurement / purchase-order process – ordering, POs, and vendor payment stay exactly as today; the module ends at the pre-filled needs-parts request.

Business Rules

1. **Shortfall flows to needs-parts.** Any confirmed remainder not covered by owned stock pre-fills the needs-parts request; the technician performs the status change.
2. **The transfer flow stays manual and unchanged.** When a technician collects internal stock, they record it in the existing transfer flow – open it, select the source, select the part received, and submit so inventory adjusts; Parts Finder neither automates nor changes this step.
3. **No procurement changes.** The module ends at the pre-filled request; ordering and payment are unchanged.

Exceptions

- Part not found anywhere → the needs-parts pre-fill covers the full quantity; the technician sets the status and the office orders.

Acceptance Criteria

- An uncovered quantity produces a pre-filled needs-parts request; the technician performs the status change.
- The existing manual transfer flow and the office procurement flow are unchanged.

7. AI & Data Governance

This section governs all three use cases: how the AI layer is run, where ClimatePros data may go, and the human gate in front of every automated action.

Model strategy.

- **Privacy-first.** ClimatePros' self-hosted private model is the priority production runtime. Commercial models may be used selectively, through a routing layer, only when a task requires it and only on data cleared for it — always within ClimatePros' data-privacy commitments. During development, external models are used against staging and mock data only.
- **Provider-agnostic build.** Every AI capability sits behind a routing layer, so the mix between the private runtime and any commercial model is a configuration decision, not an architectural one.
- **Private-runtime readiness (sign-off critical — the single largest delivery risk).** No integration test has yet been run against ClimatePros' private runtime. ClimatePros to confirm the endpoint and credentials, the exact model and version, whether any company-context / retrieval layer exists on it today, and a target date for first integration testing.

Data boundary.

- **Deterministic scrubbing (non-AI).** Personal information and pricing are scrubbed by a code-level layer before every model call and over every output, on any runtime — a placeholder substitution, not a deletion; the held values are re-inserted into the office-facing output afterward, so no customer data reaches the runtime while the record keeps the detail it needs.
- **What is scrubbed:** customer and site-contact names and contact details; part prices and vendor pricing; technician hourly rate, overhead rate, and overhead percent; and internal part numbers (excluded from the customer-facing note without exception). *(ClimatePros to confirm this list is complete for its customer commitments.)*
- **Provider-agnostic handling.** The same scrub applies regardless of which runtime serves a request, so a routing decision can never widen the data boundary.

Human-in-the-loop gates.

- **Auto Dispatch (A):** technician acceptance + dispatcher override + master switch; the system performs exactly one status transition, and only on acceptance.
- **Closeout Copilot (B):** the technician reviews, edits, and submits every note; the completeness gate prompts and never blocks — the one exception being a missing refrigerant form.
- **Parts Finder (C):** technician confirmation before any source query, and call-verification before driving; every status change is the technician's.
- **Across all three:** the system never invents and never authors — it standardises, ranks, and predicts only from data already entered or recorded, and no automated action reaches a customer site unreviewed.

Explainability, degradation, and confidence.

- **Explainability.** Every automated dispatch decision carries plain-language reasoning (a deterministic template, no AI); suggestions show their basis.
- **Graceful degradation.** No use case hard-depends on the AI layer: dispatch falls back to the manual process via the toggle and escalation; closeout completes without standardised notes (the technician writes them, as today); parts search falls back to today's manual paths.
- **One shared definition of "low confidence."** Across all three, weak signal degrades to silence, never to a wrong assertion; the exact per-feature thresholds are tuned during development against real data.

Capacity, language, and the voice beta.

- **Model sizing / peak concurrency.** The workloads are sized for small, fast models with tightly-scoped context; peak-concurrency validation against real volume is part of the architecture deliverable before this is relied on.
- **Language coverage.** Every AI surface is built and tuned in English this phase; the surrounding app displays in the technician's chosen language exactly as today. Localised prompts and cross-lingual notes are a Phase-2 enhancement.
- **Voice beta gate.** The Closeout Copilot voice capture (B1) ships only as a declared beta, and only after a written cost-and-privacy assessment is reviewed with ClimatePros.

8. Risks & Mitigation

Risk	Mitigation
Self-hosted model readiness. No integration test has run against the private runtime; access, performance, behaviour, and any company-context layer are unverified, and capacity figures are client-stated. A late or under-performing runtime jeopardises the privacy-first posture.	Model risk pinned to a named, dated sign-off item (private-runtime readiness, §7); the provider-agnostic build keeps the runtime switch a configuration decision, not a rebuild.
Adoption and notification fatigue. The dispatch cascade, the ETA prompt, and closeout prompts succeed only if technicians respond; over-pinging breeds ignore-behaviour — and the app has no remote push today, so the delivery channel itself is a build item.	Adoption engineered and measured — one-tap interactions, a quiet window, capped snooze-aware reminders, and client-owned incentives; pilot response rates reviewed weekly. The push channel is scoped explicitly as a dependency.
Response-window tuning. Cascade and prompt timing are configurable, not fixed; poor tuning either escalates too much or waits too long for an emergency.	Windows configurable at runtime , so tuning is an operational adjustment, not a release.
Historical-corpus quality. Closeout standardisation and part prediction both draw on a corpus known to contain thin, inconsistent notes; live signal may be weaker than the demonstrations.	Corpus risk degrades to silence — weak signal produces no suggestion, and the AI only standardises what the technician entered; the Parts Finder backtest quantifies prediction quality before rollout; every correction is captured to improve the signal.
Reliance on ClimatePros' priority and queue discipline. Because the engine acts on the priority as given and does not	Queue discipline handled by people, not guesswork — the dispatcher watches System activity and can override or

Risk	Mitigation
screen the queue, mis-prioritised or non-dispatchable work left in Ready for Dispatch could be auto-processed; this rests on dispatcher oversight, the master switch, and ClimatePros keeping the queue and priorities accurate.	escalate any dispatch; ClimatePros' ownership of classification and queue hygiene is called out for sign-off.
Third-party surfaces. The United gateway, the mapping / traffic service, the GPS / location feed, and the check-in event sit outside techjays' control; the GPS locator is an open, ticketed defect.	Third-party surfaces wrapped — the vendor abstraction isolates the United contract; a gateway or feed failure degrades a single source or tier, never the whole flow; GPS gaps fall back to last-known position with a "not live" marker.
Dual-scheduler overlap. ClimatePros' overnight scheduler and Auto Dispatch share the FieldJetX database; an unconfirmed run mode could produce confusing states if the boundary isn't held.	Scheduler boundary held in writing — the shared database as the single source of truth, "a manual assignment always wins," and toggle-on / off ownership (the engine owns the in-scope queue while on, the scheduler while off) keep the two from colliding.
Scope expansion. The remaining dispatch statuses and the deferred features are the standing expansion pressure.	Strict scope governance — the per-use-case out-of-scope lists and the Future Enhancements section give every expansion request a home that is not this phase.

Table 2. Risks and mitigations

9. Success Measures

Each measure below is a target agreed with ClimatePros, with its exact computation confirmed at that point. Targets can be tracked on a pilot area or group once the system is in day-to-day use.

Use Case	Success Measures
Auto Dispatch (A)	the share of dispatches auto-assigned within the cascade without escalation; the share arriving on site within the confirmed response window; and how often the dispatcher has to step in and override — each against the pre-launch manual baseline.
Closeout Copilot (B)	median closeout time versus the pre-rollout baseline (targeting up to roughly half); the share of standardised notes accepted with little or no editing; and the completeness-gate pass rate (fewer jobs bounced back by the office).
Parts Finder (C)	an offline backtest of part prediction against historical jobs (a top-3 hit-rate against the parts actually used), reported before rollout; and, during the pilot, the share of parts-sourcing sessions resolved without a return visit.

Table 3. Success measures by use case

10. Future Enhancements

Auto Dispatch: the remaining dispatch statuses beyond Ready for Dispatch; automated eligibility screening of the queue; returning-work / call-back routing to the technician who last worked the job; same-site batching of multiple dispatches; skill- and seniority-aware ranking, including holding a senior technician in reserve; fatigue / hours-on-duty weighting; after-hours / weekend / on-call mode; multi-technician / helper sourcing; per-site technician preferences and exclusions; offering across a neighbouring area; site access-hour gating.

Closeout Copilot: prompting for root cause during the job (this phase captures it at closeout); cross-visit prompting; detecting which forms a job requires (beyond the refrigerant form); client-configurable note layouts; voice capture general availability; interpreting the content of photo / video attachments.

Parts Finder: cost-based ranking / price optimisation (server-side); cross-branch warehouse collection; additional vendors via the unified vendor middleware; per-equipment parts history in the app; the shopper / runner delivery program.

Cross-cutting: multilingual AI surfaces; deeper interplay with the overnight scheduler; operational dashboards for dispatch, closeout, and parts; and — a direction ClimatePros flagged — extending the same capabilities to **HVAC / duct-work and rooftop-unit** work as that side of the business grows.